
Software Qualities: Thread Safety

Marco Faella

A class is *thread-safe* if...

...multiple threads
can use its objects concurrently
with no client-side synchronization

The class takes care of synchronization, *when needed*



*StringBuffer and Vector are thread safe,
StringBuilder and ArrayList are not*

Between a rock and a hard place

- **Race conditions**

- Threads step on each other
- Missing synchronization

- **Deadlocks**

- Threads get stuck waiting for each other
- Too much (undisciplined) synchronization

Concurrency policies

Establish **what can happen concurrently**

The rest must be *serialized*

Example:

- Employee class, methods getSalary and setSalary
- Concurrency policy:
 1. Method calls on **different employees** can run **concurrently**
 2. Method calls on **the same employee** must be **serialized**

Object-level concurrency

Concurrency policies

Establish **what can happen concurrently**

The rest must be *serialized*

Common concurrency policies:

More concurrency ↓	Name	What is concurrent?	How many locks?
	Class-level	Access to different classes	One lock per class
	Object-level	Access to different objects	One lock per object
	Method-level	Access to different methods	One lock per method of each object
	Anarchy	Everything	No locks



What about water containers?

A concurrency policy for water containers

- If containers a and b are **not** connected, any method on a can run **concurrently** with any method on b
 - Exception: `a.connectTo(b)` and `b.connectTo(a)` must be serialized
- Any other pair of method invocations must be **serialized**
- So, one lock per **group**
 - However, groups change in time...

Recall Speed1

Containers have a single field:

```
Group group = new Group(this);
```

Group is a nested class:

```
static class Group {  
    double amountPerContainer;  
    Set<Container> members;  
    ...  
}
```

Thread-safe connectTo?

```
public void connectTo(Container other) {  
    synchronized (this.group) {  
        synchronized (other.group) {  
            ... The actual operation here  
        }  
    }  
}
```

Deadlock!

Thread1:
a.connectTo(b)

Thread2:
b.connectTo(a)

Two ways to prevent deadlocks

1. Acquire monitors while holding a *global* monitor
2. Acquire monitors in a *fixed order*

First solution: Global monitor v1

A single monitor for the entire class

```
static Object globalLock = new Obj  
  
public void connectTo(Container other)  
    synchronized (globalLock) {  
        synchronized (this.group) {  
            synchronized (other.group) {  
  
                ... The actual operation here  
  
            }  
        }  
    }  
}
```

Wrong!

First solution: A *proper* global monitor

An *explicit* lock

```
static ReentrantLock globalLock = new ReentrantLock();

public void connectTo(Container c) {
    globalLock.lock();
    synchronized (this.group) {
        synchronized (other.group) {
            globalLock.unlock();
            ... The actual connection ...
        }
    }
}
```

But...

No deadlock

Problems with the global monitor

1. Introduces a bottleneck

- More serialization than strictly necessary
- Slows down even single-threaded execution

2. Race condition (explained later)

Second solution: Fixed-order locking

Add a **unique ID** to groups:

```
static class Group {
```

Nested class of Container

```
    double amountPerContainer;
```

```
    Set<Container> elems = new HashSet<>();
```

```
    final int id = nGroups.incrementAndGet();
```

```
    static final AtomicInteger nGroups = new AtomicInteger();
```

```
    ...
```

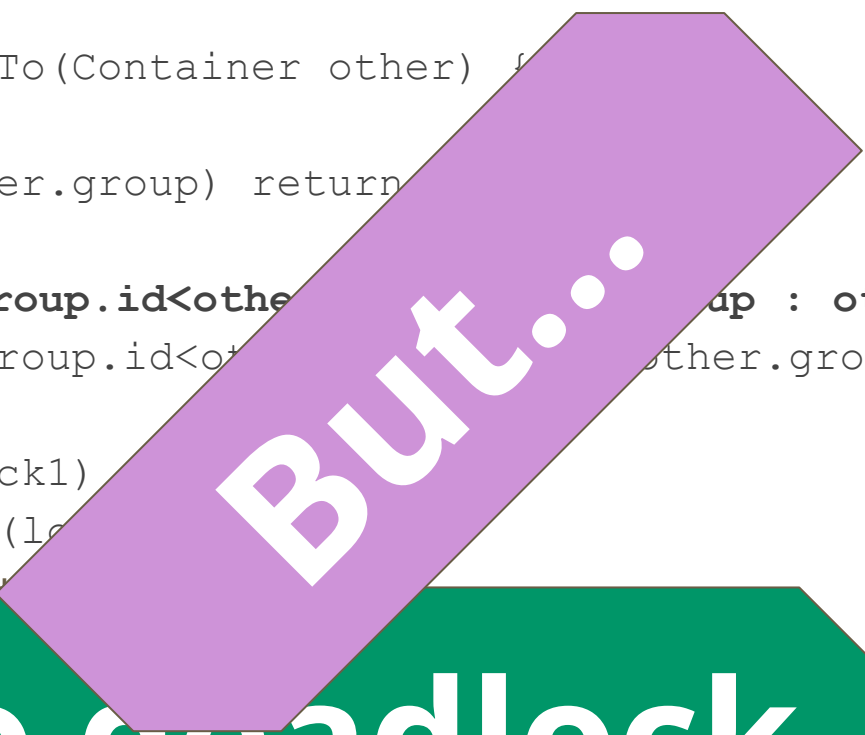
```
}
```

A class of mutable
thread-safe integers

Second solution: Fixed-order locking (2)

Acquire monitors **in the order of their IDs**:

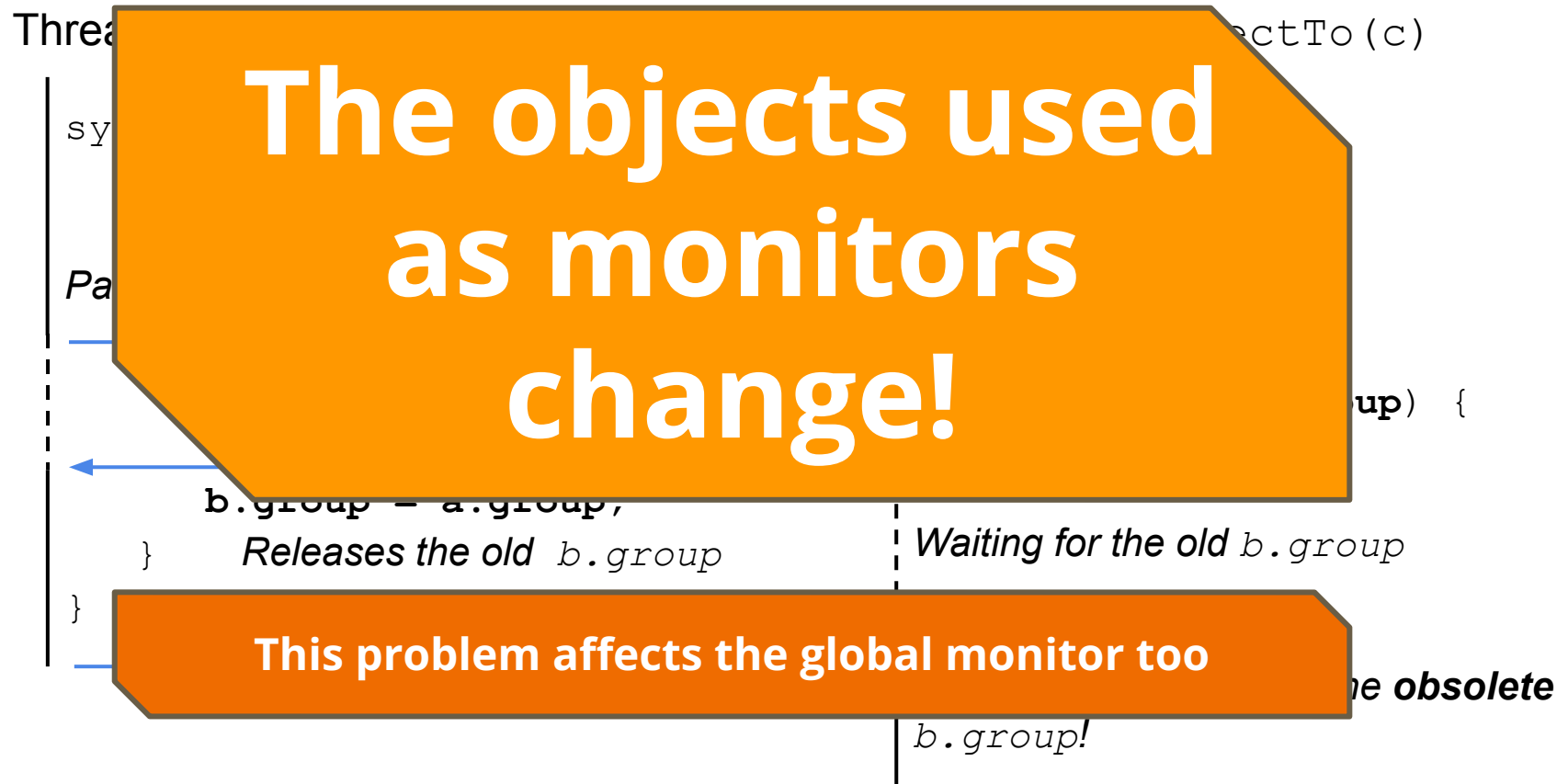
```
public void connectTo(Container other) {  
    if (group == other.group) return;  
  
    Object lock1 = group.id < other.group ? other.group : other.group,  
        lock2 = group.id < other.group ? other.group : group;  
  
    synchronized (lock1)  
        synchronized (lock2)  
            Merge group;  
}  
}
```



No deadlock

A hidden race condition

Assume: three unconnected containers `a`, `b`, `c`
`a.group.id < b.group.id < c.group.id`



Full serialization v1

A single explicit lock for the whole class

```
static ReentrantLock globalLock = new ReentrantLock();
```

```
public void connectTo(Container other) {
```

```
    globalLock.lock();
```

```
    synchronized (this.group) {
```

```
        synchronized (other.group) {
```

```
            ... The actual operation here
```

```
        }
```

```
    }
```

```
    globalLock.unlock();
```

```
}
```

These are useless now!

Full serialization v2

No deadlock

```
static Resource r = new Resource();
```

```
public void
```

```
    globalLock.lock();
```

```
    ... Th
```

```
    global
```

```
}
```

No race condition

No concurrency!

That



Lock-free synchronization

Attempting something multiple times,
until no contention is detected

*It reminds me of the CPU instruction
Compare-And-Swap (CAS)*



Compare-And-Swap instruction

CAS (src: address, dest: address, old: value)

Atomically perform the following operation:

```
if the current value of dest is old then
    swap contents of src and dest
    return true
else
    return false
```

x86: Compare and
exchange (CMPXCHG)

CAS-based increment

Atomic “n++” without locks (C syntax):

```
do {  
    int old = n  
    int src = old + 1  
    success = CAS(&src, &n, old)  
} while (!success)
```

Thread-safe connections

```
public void connectTo(Container other) {  
  
    while (true) {  
        Object lock1 = group.id<other.group.id ? group : other.group,  
            lock2 = group.id<other.group.id ? other.group : group;  
  
        if (lock1 == lock2) return; // Already connected  
  
        synchronized (lock1) {  
            synchronized (lock2) {  
  
                if ((lock1 == group && lock2 == other.group) ||  
                    (lock2 == group && lock1 == other.group)) {  
                    ... // Locks are current: merge groups and exit  
                }  
  
                // At least one lock is obsolete: release both and try again  
            }  
        }  
    }  
}
```

Unbounded number of attempts

Respect order of ids

Check if locks are *current*

Summary

Speed1

not thread-safe



“Small” global monitor

bottleneck, race condition



Fixed-order locking

race condition



“Large” global monitor: Full serialization

no concurrency!



Trying multiple times

thread-safe

Thread safety: conclusions

1. Establish your **concurrency policy**
2. Avoid using non-constant expressions as monitors
3. Sometimes you have a choice between:
 - Full serialization
 - Trying multiple times (as in *lock-free synchronization*)
4. Low-level synchronization is tricky: **avoid it** whenever possible!